Object-Oriented and Classical Software Engineering

8th Edition, WCB/McGraw-Hill, 2010

Original by:
Stephen R. Schach
srs@vuse.vanderbilt.edu

Updated by:
Nguyen Manh Hung - PTIT
Do Thi Bich Ngoc - PTIT

CHAPTER 10

REQUIREMENTS

Overview

- Determining what the client needs
- Overview of the requirements workflow
- Understanding the domain
- The business model
- Initial requirements
- Case study: The Hotel Reservation Management (HRM)

The Aim of the Requirements Workflow

- To answer the question:

  What must the product be able to do?

Determining What the Client Needs

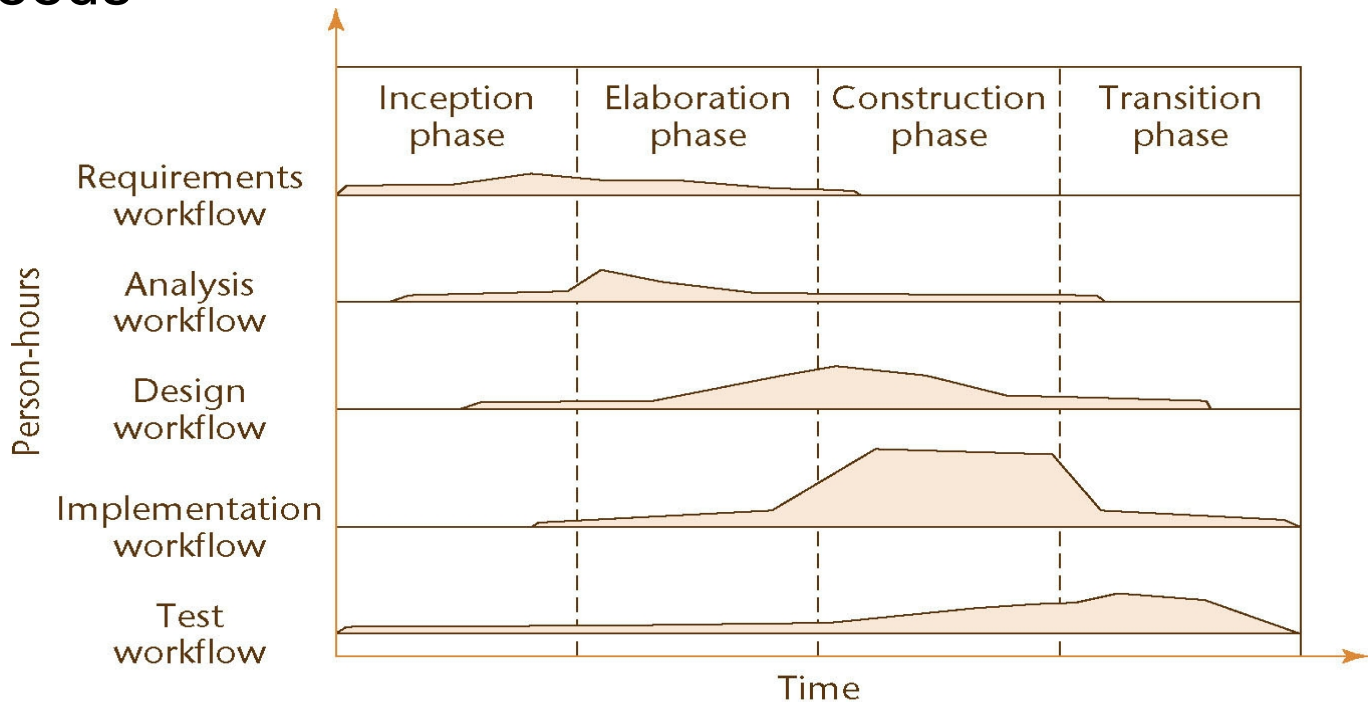
- Misconception
  - We must determine what the client wants
- “I know you believe you understood what you think I said, but I am not sure you realize that what you heard is not what I meant!”
- We must determine what the client *needs*

Determining What the Client Needs (contd)

- It is hard for a systems analyst to visualize a software product and its functionality
  - The problem is far worse for the client
- A skilled systems analyst is needed to elicit the appropriate information from the client
- The client is the only source of this information

Determining What the Client Needs (contd)

- The solution:
  - Obtain initial information from the client
  - Use this initial information as input to the Unified Process
  - Follow the steps of the Unified Process to determine the client's real needs



Overview of the Requirements Workflow

- First, gain an understanding of the *application domain* (or *domain*, for short)
  - The specific environment in which the target product is to operate
- Second, build a business model
  - Model the client's business processes
- Third, use the business model to determine the client's requirements
- Iterate the above steps

Definitions

- Discovering the client's requirements
  - R*equirements elicitation* (or *requirements capture*)
  - Methods include interviews and surveys
- Refining and extending the initial requirements
  - *Requirements analysis*

Understanding the Domain

- Every member of the development team must become fully familiar with the application domain
  - Correct terminology is essential
- Construct a glossary
  - A list of technical words used in the domain, and their meanings

Business Model

- A *business model* is a description of the business processes of an organization
- The business model gives an understanding of the client's business as a whole
  - This knowledge is essential for advising the client regarding computerization
- The systems analyst needs to obtain a detailed understanding of the various business processes
  - Different techniques are used, primarily interviewing

Interviewing

- The requirements team meet with the client and users to extract all relevant information

Interviewing (contd)

- There are two types of questions
  - *Close-ended* questions require a specific answer
  - *Open-ended* questions are posed to encourage the person being interviewed to speak out
- There are two types of interviews
  - In a *structured* interview, specific preplanned questions are asked, frequently close-ended
  - In an *unstructured* interview, questions are posed in response to the answers received, frequently open-ended

Interviewing (contd)

- Interviewing is not easy
  - An interview that is too unstructured will not yield much relevant information
  - The interviewer must be fully familiar with the application domain
  - The interviewer must remain open-minded at all times
- After the interview, the interviewer must prepare a written report
  - It is strongly advisable to give a copy of the report to the person who was interviewed

Other Techniques

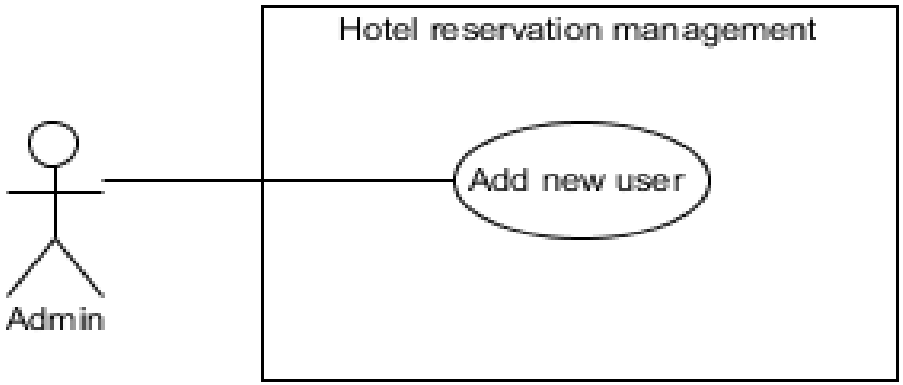
- Interviewing is the primary technique
- A questionnaire is useful when the opinions of hundreds of individuals need to be determined
- Examination of business forms shows how the client currently does business

Other Techniques (contd)

- Direct observation of the employees while they perform their duties can be useful
  - Videotape cameras are a modern version of this technique
  - But, it can take a long time to analyze the tapes
  - Employees may view the cameras as an unwarranted invasion of privacy

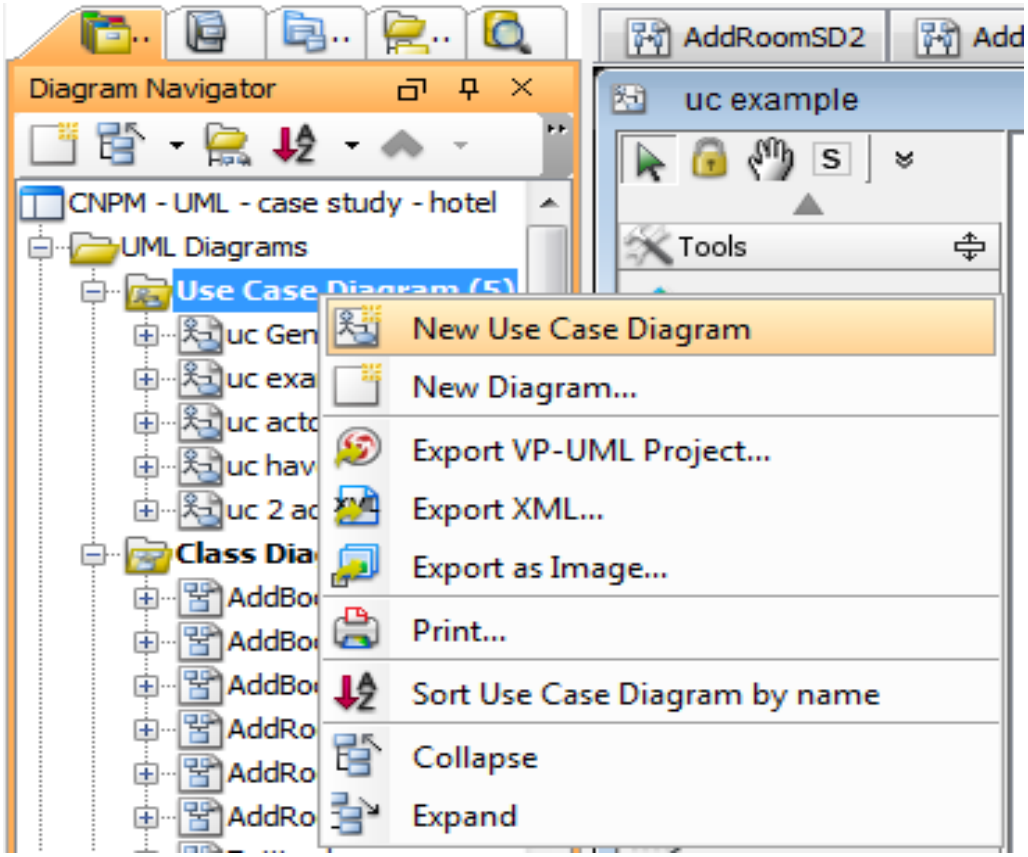
Use Cases

- A use case models an interaction between the software product itself and the users of that software product (*actors*)
- Example:



Use Cases (contd)

- In VP, select new use case diagram:



Actor

- An actor is a member of the world outside the software product
- It is usually easy to identify an actor
  - An actor is frequently a user of the software product
- In general, an actor plays a role with regard to the software product. This role is
  - As a user; or
  - As an initiator; or
  - As someone who plays a critical part in the use case

Actor (contd)

- A user of the system can play more than one role
- Example: A customer of the bank can be
  - A **Borrower** or
  - A **Lender**

Actor (contd)

- Conversely, one actor can be a participant in multiple use cases

- Example: A **Borrower** may be an actor in
  - The `Borrow Money` use case;
  - The `Pay Interest on Loan` use case; and
  - The `Repay Loan Principal` use case

- Also, the actor **Borrower** may stand for many thousands of bank customers

Actor (contd)

- An actor need not be a human being
- Example: An e-commerce information system has to interact with the credit card company information system
  - The credit card company information system is an actor from the viewpoint of the e-commerce information system
  - The e-commerce information system is an actor from the viewpoint of the credit card company information system

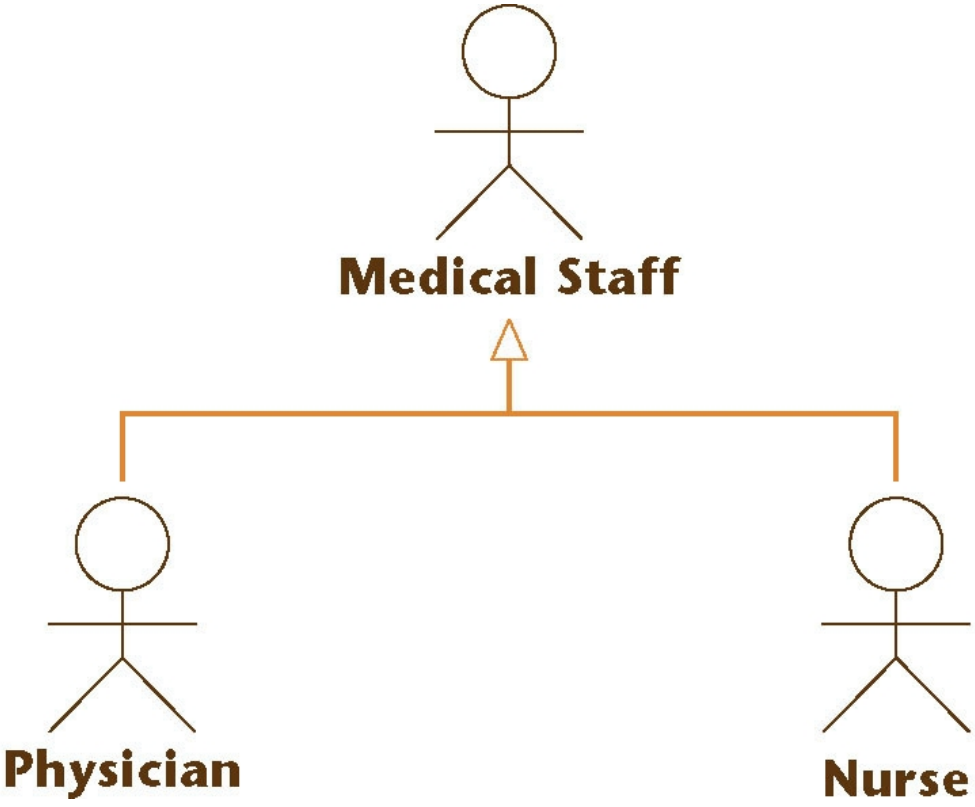
Actor (contd)

- A potential problem when identifying actors
  - Overlapping actors

- Example: Hospital software product
  - One use case has actor **Nurse**
  - A different use case has actor **Medical Staff**
  - Better:
    - Actors: **Physician** and **Nurse**

Actor (contd)

- Alternatively:
  - Actor **Medical Staff** with two specializations: **Physician** and **Nurse**

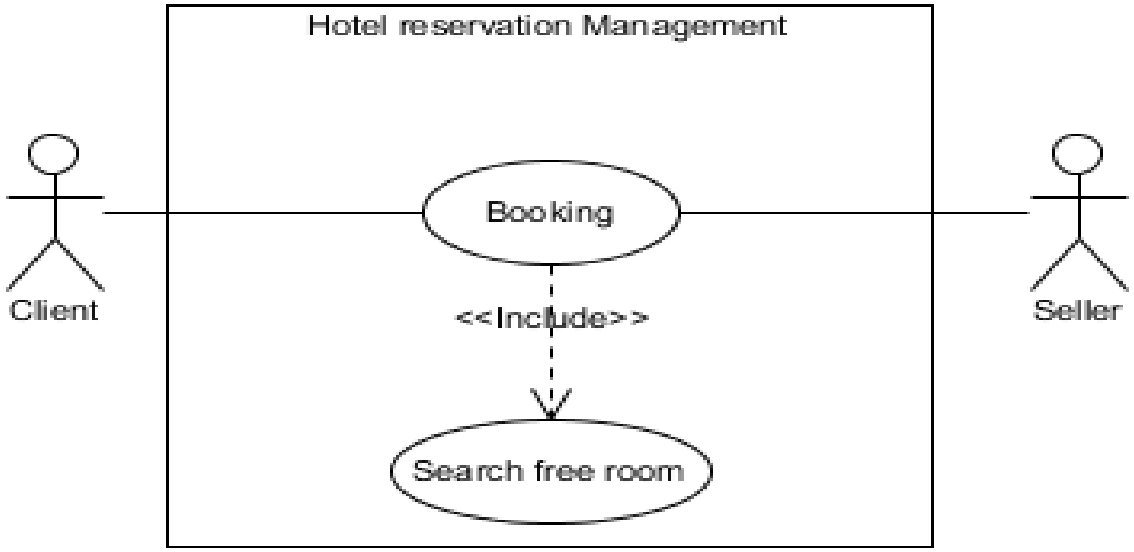


Use case (contd)

- Three relationships between use cases:
  - Include
  - Extend
  - Genelization

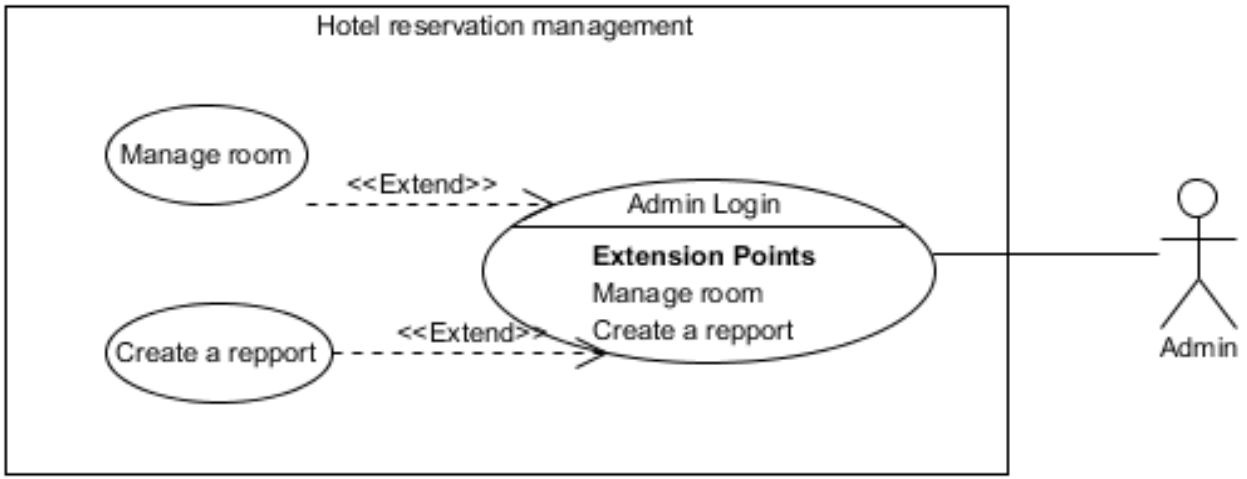
Use case (contd)

- Include relationship:
  - UC A *includes* UC B if action B is a part of action A
  - Each time realise A, the actor also has to realise B
  - If not finish B, the actor can not finish A.
- Ex: booking a room includes searching for available room.



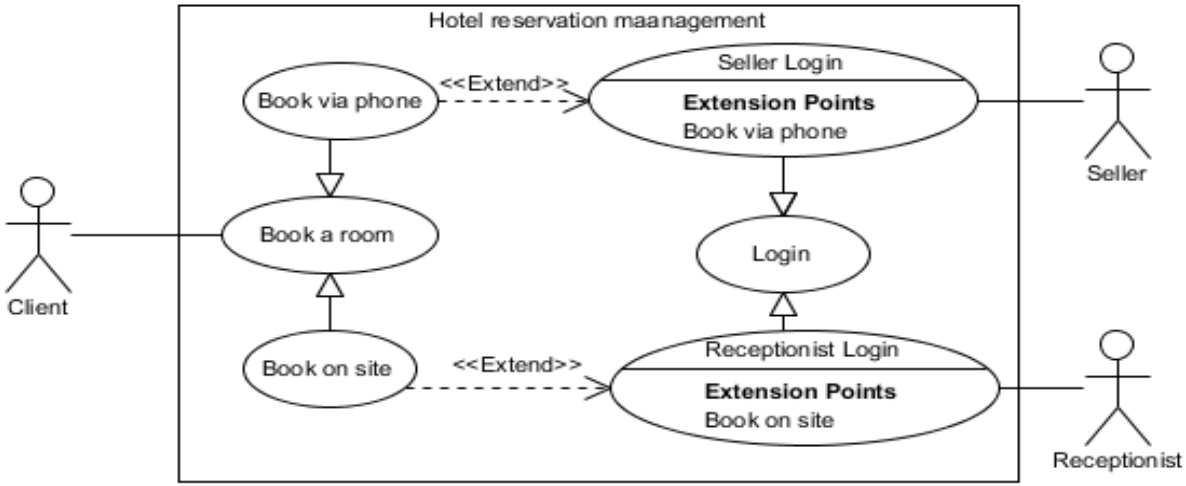
Use case (contd)

- Extend relationship:
  - UC B *extends* from UC A if action B could be relalised from action A
  - Doing B is optional while doing A
  - If not finish B, the actor can still finish A.
- Ex: Manage a room or view repport are extened from login of admin.



Use case (contd)

- Genelization relationship:
  - UC B *inherites* from UC A if action B is an instance of action A
  - The logic of doing B is similar to that of doing A
- Ex: Booking a room via phone and booking at the counter are the same and a they are a kind of booking in genaral.



Initial Requirements

- The initial requirements are based on the initial business model
- Then they are refined
- The requirements are dynamic — there are frequent changes
  - Maintain a list of likely requirements, together with use cases of requirements approved by the client

Initial Requirements (contd)

- There are two categories of requirements
- A *functional requirement* specifies an action that the software product must be able to perform
  - Often expressed in terms of inputs and outputs
- A *nonfunctional requirement* specifies properties of the software product itself, such as
  - Platform constraints
  - Response times
  - Reliability

Initial Requirements (contd)

- Functional requirements are handled as part of the requirements and analysis workflows
- Some nonfunctional requirements have to wait until the design workflow
  - The detailed information for some nonfunctional requirements is not available until the requirements and analysis workflows have been completed

HOTEL RESERVATION MANAGEMENT SYSTEM

Requirement process

- Concept exploration
  - Collect terms/concepts in the domain
  - Build the glossary list
- Business model by natural language
  - Scope and objective
  - Who can use the software? What are functions that an actor can do?
  - How does each function work?
  - Which information/object the system need to process?
  - Relationship among objects?

Requirement process (contd)

- Business model by technical language - UML
  - Identify actors of the system
  - Identify use cases for each actor
  - Refine the general use case diagram of the system
  - Detail use case diagram for each function

HRM: concept exploration

- Step 1: Use *brain storming* to discover terms/concepts in the application domain
  - Ex: room, single room, double room, booking...
- Step 2: Build the glossary list
  - Grouped by its category
  - Indexed continuously among groups

No.	Terms in your language	Terms in English	Meaning
Terms related to the objects in the hotel			
1	Phòng đơn	Single room	A hotel room which has a single bed for an adult customer
2	Phòng đôi	Double room	A hotel room which has a double bed for two adult customers
Terms related to user			
3	Người quản lí	Manager	...
4	NV lễ tân	Receptionist	...
....			

HRM: BM by natural language

- Objective
  - A desktop application enablings to manage the reservation of a single hotel

HRM: BM by natural language (contd)

- Scope
  - Used within a hotel (could be on many computers)
  - Only following employees of the hotel could use:
    - Manager
    - System admin
    - Seller
    - Receptionist
  - Only following functions are enabled:
    - Management of hotel, room, user information
    - Booking room, cancel booking,
    - Checkin, checkout and payment
    - View statistical repports related to booking
  - The users which are not mentioned have no right to use
  - The functions which are not mentioned are not available

HRM: BM by natural language (contd)

- Who could do what in the system?
  - Manager
    - Manage hotel infor: add, edit, delete, search and view
    - Manage room infor: add, edit, delete, search/view
    - View statistical repports: income by room/client/month
  - Admin (by the requirement form the related user)
    - Manage user infor: add, edit, delete, search/view
  - Seller (by the requirement of customers)
    - Search and book room via phone
    - Cancel room via phone
  - Receptionist (by the requirement of customers)
    - Search and book room at the counter desk
    - Cancel room at the counter desk
    - Checkin for a client
    - Checkout/ payment process for a client

HRM: BM by natural language (contd)

- How does each function work?
  - Edit a room
    - A manager logged in → Manager home UI shown with menu to select to: manage hotel, manage room, view statistic
    - Select manage room → Room management UI shown with menu to select to: add, edit, delete, search
    - Select edit room → Room search UI shown with a textbox to enter keyword, a earch button
    - Enter a keyword + click search → Search results UI shown with a list of room whose name contains the entered keyword. Each raw associated to buttons of edit, delete

HRM: BM by natural language (contd)

- How does each function work?
  - Edit a room (contd)
    - Select edit a row → Edit room UI shown with textfields: name, type, price, description which are already filled by the information of the selected room, buttons: save, reset.
    - Edit some fields + click save → The successful message shown.
    - Click OK of the message → Return to the Manager home UI.

HRM: BM by natural language (contd)

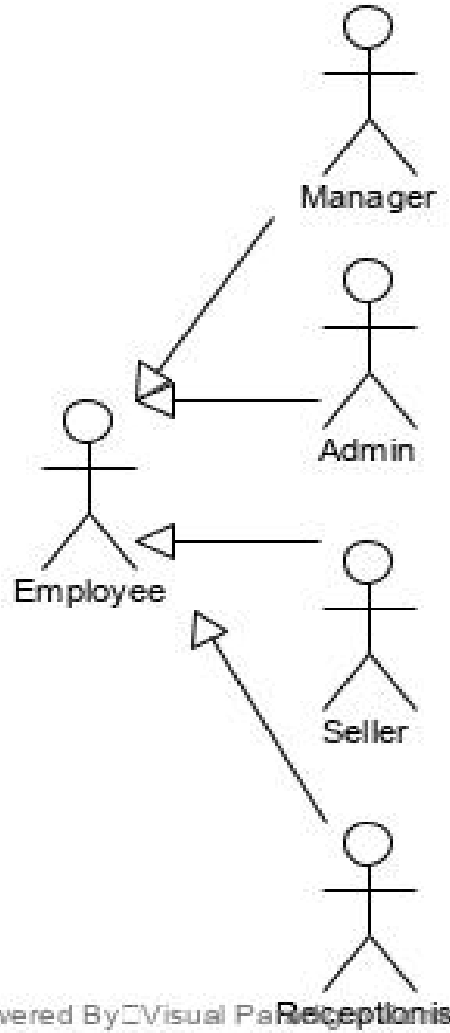
- Objects need to be processed
  - Hotel:
    - Name, address, star rating, décription
  - Room:
    - Name, type, price, description
  - User:
    - Username, password, fullname, role
  - Client:
    - Name, address, tel, email, note
  - Bill:
    - Client's infor, Creator's infor, booking infor, service infor
  - Service:
    - Name, category, unity, price, description

HRM: BM by natural language (contd)

- Relationships among objects
  - A hotel could have 1 or many rooms
  - A client could book many times, each time many rooms
  - A room could be booked many time at different periods
  - A service could be used many time in each booked room
  - Each booked time, a room could use 0 or many service
  - Each booked time, a client could pay many times, each time receives a bill
  - ...

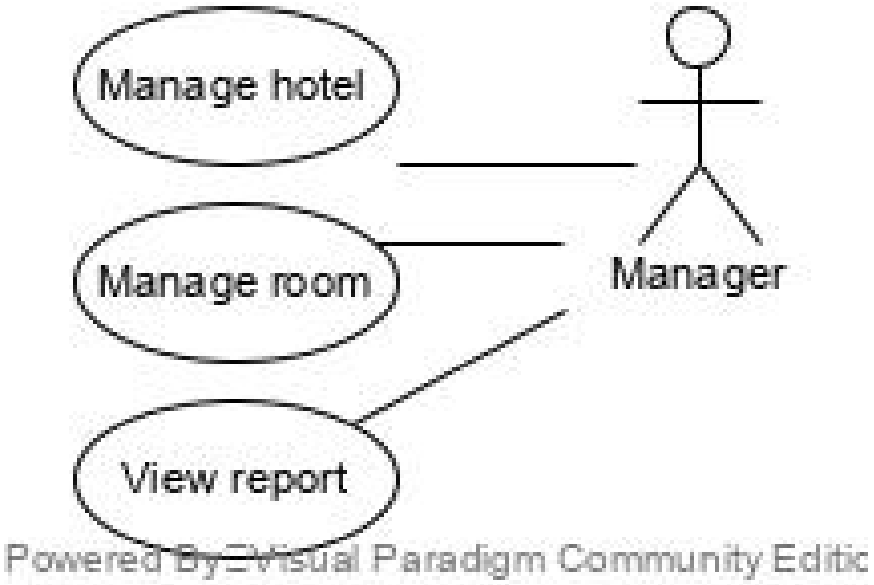
HRM: BM by technical language - UML

- Identification of actors
  - Manager
  - Admin
  - Seller
  - Receptionist
  - Thay are hotel employee → create an abstract actor of hotel employee
  - Client : indirect actor



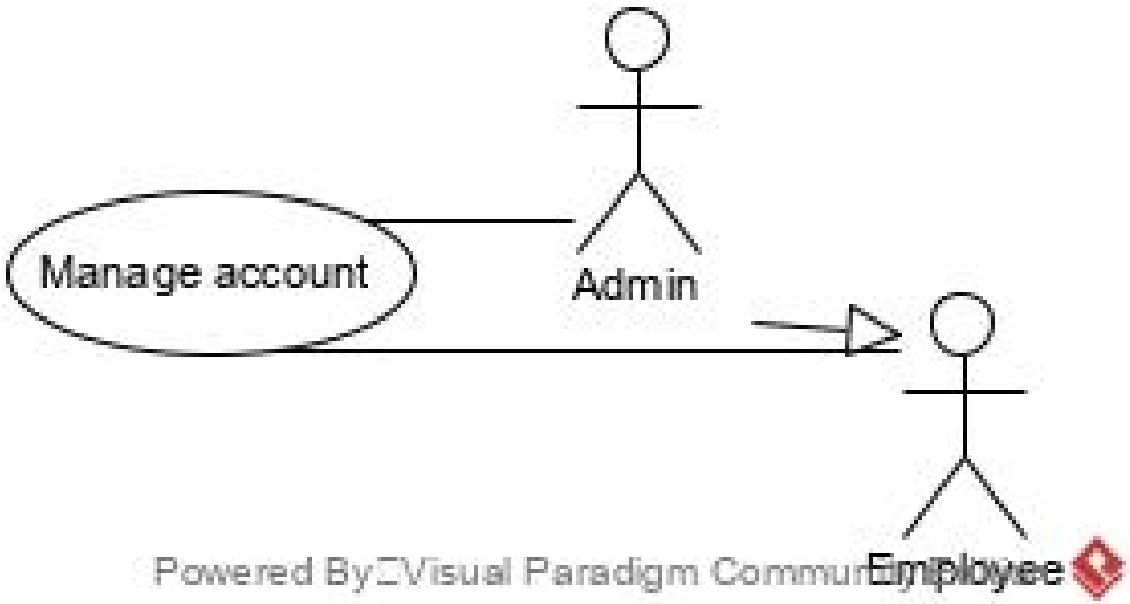
HRM: BM by technical language - UML(contd)

- Identification of use case
  - Manager
    - Manage hotel infor
    - Manage room infor
    - View statistics



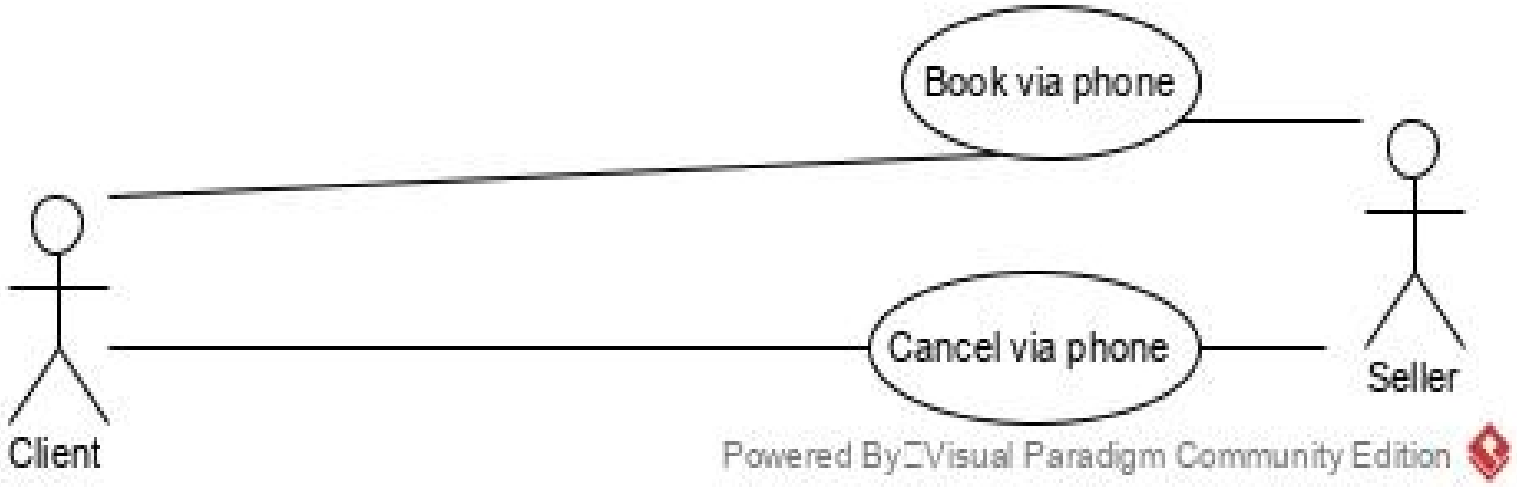
HRM: BM by technical language - UML(contd)

- Identification of use case
  - Admin
    - Manage user account (by the user's requirement)



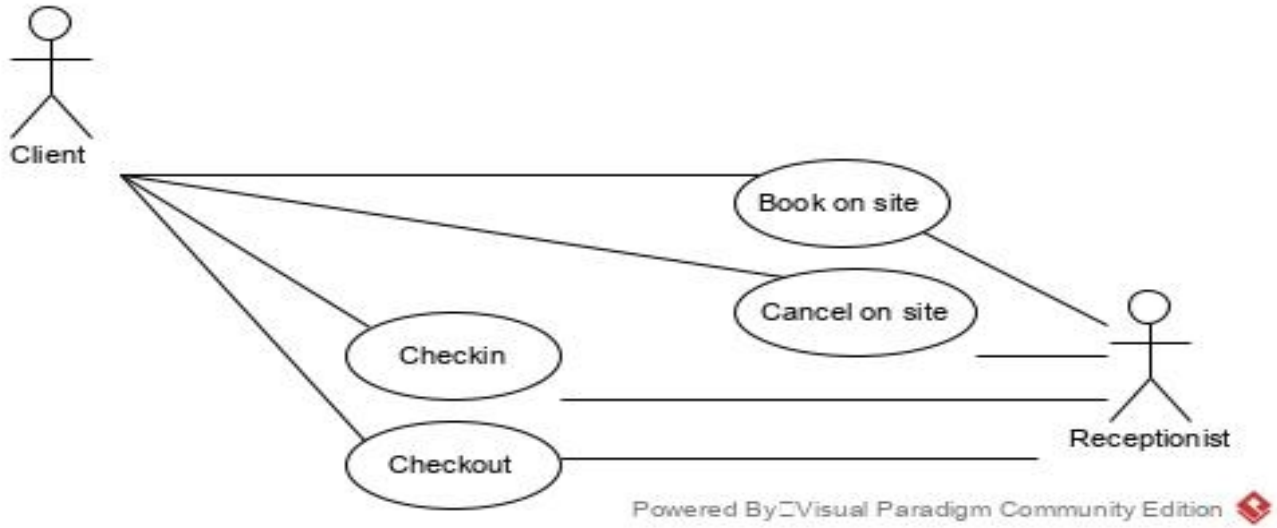
HRM: BM by technical language - UML(contd)

- Identification of use case
  - Seller
    - Booking via phone (by the client's requirement)
    - Cancel via phone (by the client's requirement)



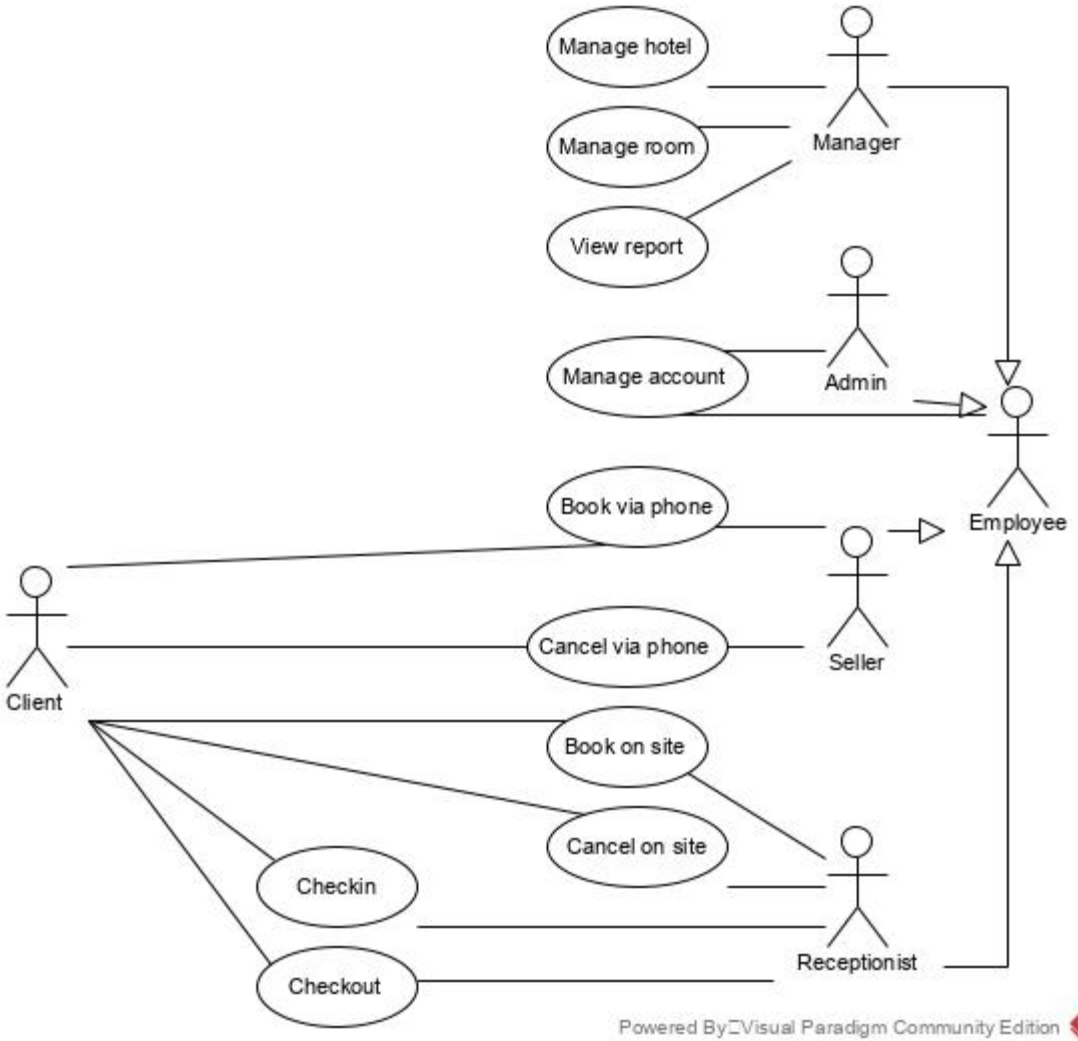
HRM: BM by technical language - UML(contd)

- Identification of use case
  - Receptionist
    - Booking on site (at the counter by the client's requirement)
    - Cancel on site (at the counter by the client's requirement)
    - Checkin (by the client's requirement)
    - Checkout (by the client's requirement)



HRM: BM by technical language - UML(contd)

- Identification of use case
  - All initial use cases

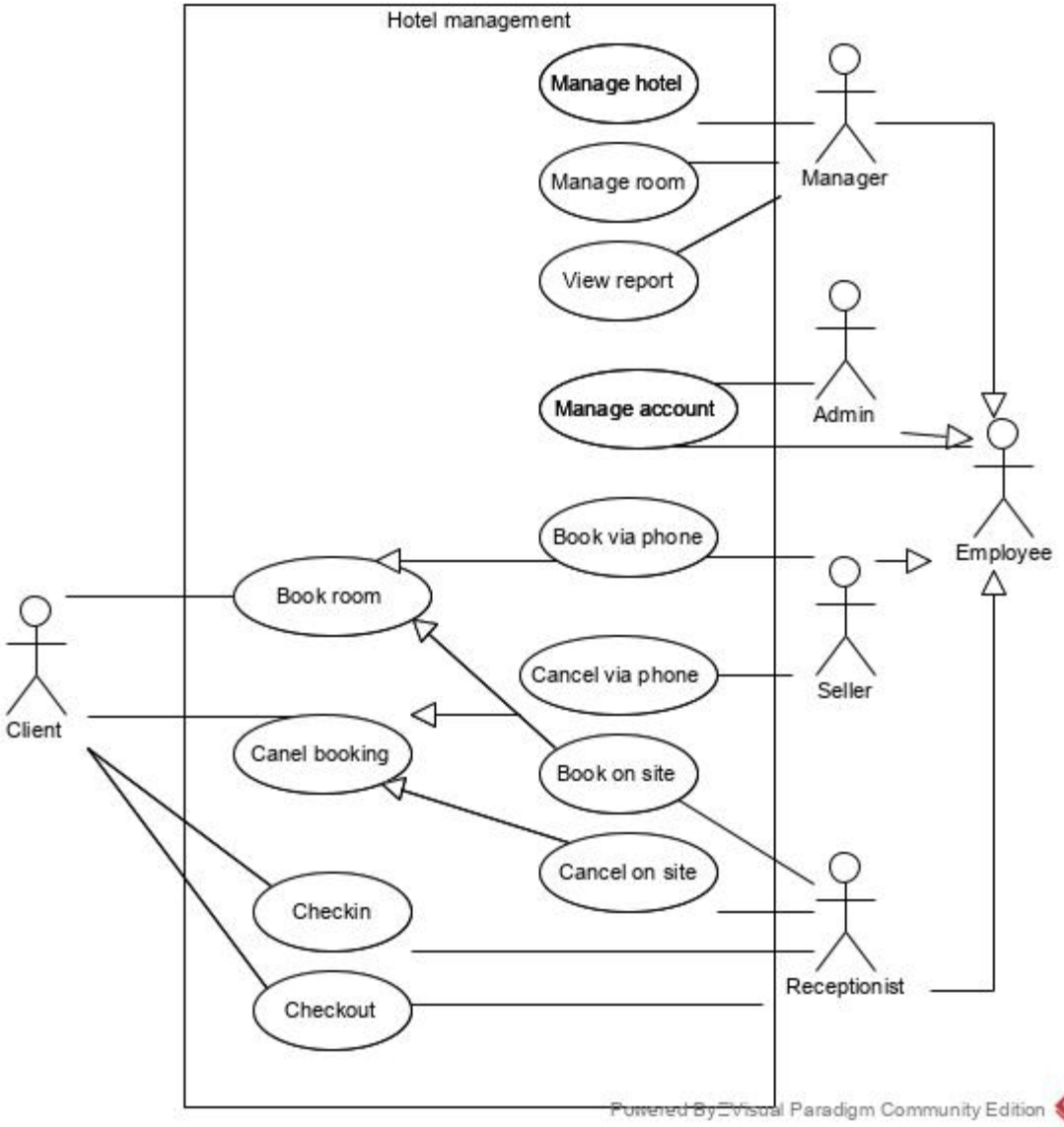


HRM: BM by technical language - UML(contd)

- Refine use case
  - Booking via phone and booking on site are the same → create an abstract UC of Booking
  - Cancel via phone and cancel on site are the same → create an abstract UC of Cancel

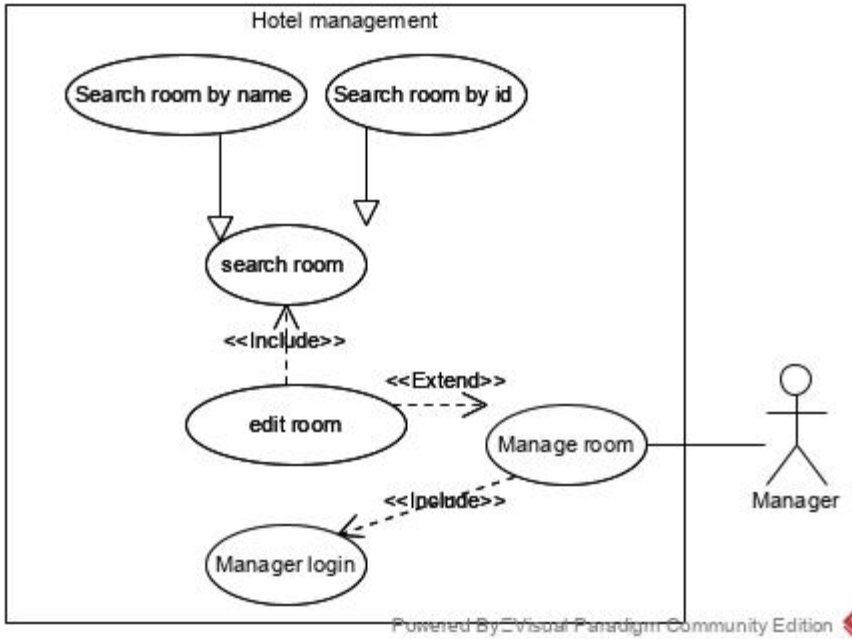
HRM: BM by technical language - UML(contd)

- Refine use case
  - General UC diagram



HRM: BM by technical language - UML(contd)

- Detail use case
  - Edit room
    - Is a sub UC of manage room (optional select from it) → extend from manage room
    - Must login (for all manage room) → include by manage room
    - Must search room to edit → include by edit room

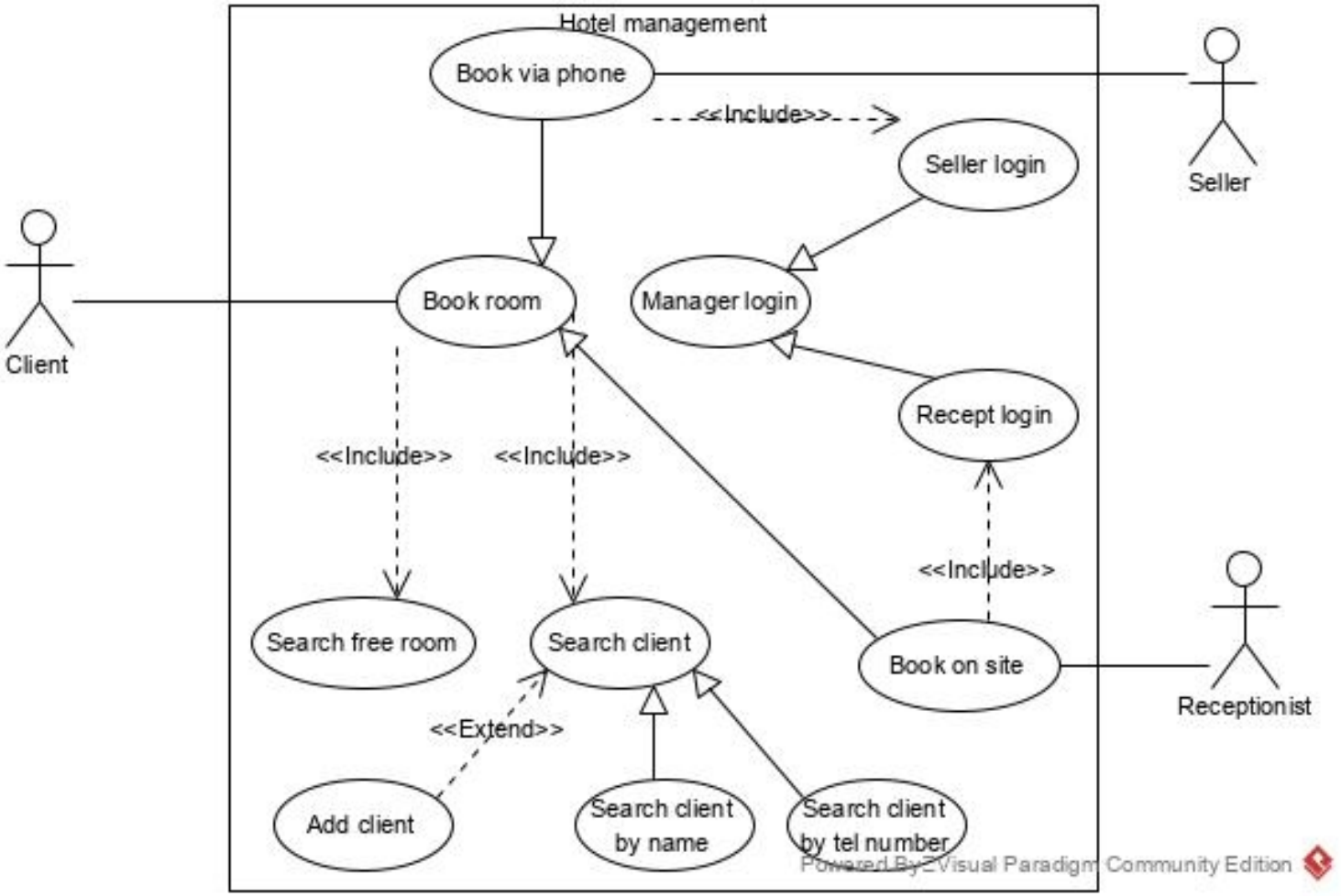


HRM: BM by technical language - UML(contd)

- Detail use case
  - Booking room
    - Reuse the relationships from reneral diagram
    - Must login (seller or receptionist) → include
    - Must search available room → include
    - Must search client → include
    - If new client → add new (sometimes only) → extend

HRM: BM by technical language - UML(contd)

- Detail use case
  - Booking room



HRM: BM by technical language - UML(contd)

- Detail use case
  - … continue with all remain use cases

End of requirement

(except review and correct errors)